

Name of the Student	P Surendra Kumar Reddy
Reg. No	192425224
GitHub Link	https://github.com/suri307/MLA0403-DEEP-LEARNING/tree/main/ASSESSMENTS

SIMATS ENGINEERING

CO5 - ASSESSMENT TOOL 1

Design Desk

COURSE NAME: Deep Learning
SLOT :A

COURSE CODE: MLA0403
Faculty : Dr.Arul Raj A.M

COURSE OUTCOME COVERED

CO5: Students will be able to analyze and explain advanced generative deep learning models including Autoencoders, Deep Belief Networks, Boltzmann Machines, Deep Boltzmann Machines, and Generative Adversarial Networks. They will be able to compare their architectures, explain their learning mechanisms, and apply suitable generative models to real-world problems. (BTL-3)

Course Outcome Weightage: 35%
Assessment Tool Weightage: 30%

Duration: 90 minutes
Mark : 25

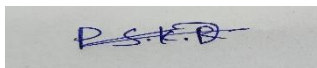
Code of Conduct:

I P Surendra Kumar Reddy (192425224) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Name of the student: P Surendra Kumar Reddy



Criteria	Subject Knowledge and Conceptual Understanding (2)	Technical Accuracy and Application of Concepts (3)	Problem-Solving and Analytical Thinking (2)	Communication Skills and Clarity of Explanation (2)	Confidence, Presentation and Professional Behaviour (1)	Total (10)
Weightage	20 %	30%	20%	20 %	10 %	100 %
Q1						
Q2						
Q3						
Q4						
Q5						
Total						

Faculty Signature :

Total Marks:

QUESTIONS: 25

Total Marks:25

1. Design an Autoencoder-Based Feature Learning System

A manufacturing company wants to detect defective products from high-dimensional sensor data. Design an Undercomplete Autoencoder with suitable encoder, latent representation, decoder, activation functions, and reconstruction loss. Explain how the design performs dimensionality reduction and feature learning.

2. Design a Robust Regularized Autoencoder

A healthcare organization has noisy patient-record features and needs a robust representation for downstream classification. Design a Regularized Autoencoder using an appropriate regularization strategy. Explain the encoder-decoder structure, regularization mechanism, training objective, and how the design reduces overfitting and improves useful feature extraction.

3. Design a Deep Generative Model

A recommendation or content-generation system requires learning complex data distributions. Design a Deep Belief Network or Deep Boltzmann Machine using suitable visible and hidden layers. Illustrate the architecture, explain the learning process, and justify the choice of the model for the proposed application.

4. Design a Stochastic and Contractive Representation Model

A real-world image or signal-processing application requires representations that remain useful under small input variations. Design a model using Stochastic Encoders/Decoders or a Contractive Encoder. Specify the architecture, objective function, and robustness mechanism, and explain how the design improves representation stability.

5. Design a GAN-Based Data Generation System

An organization has limited training images and wants to generate realistic synthetic samples. Design a Generative Adversarial Network with a Generator and Discriminator. Specify the input, network flow, adversarial training process, loss functions, and evaluation approach. Compare the proposed GAN design with an Autoencoder and justify why GAN is suitable for the application.

Design Desk Guidelines

- Identify the real-world problem, requirements, inputs, outputs and constraints.
- Draw a clear architecture or workflow diagram for the proposed model.
- Specify important layers, units, activation functions, latent representation and loss functions.
- Explain the training or learning mechanism and justify major design choices.
- Mention suitable evaluation measures and expected outcomes.
- Use neat labeling and present the design in a logical sequence.
- The solution should be original and written in the student's own words.

Q1. Design an Autoencoder-Based Feature Learning System

1. Problem Understanding

A manufacturing company needs to detect defective products using **high-dimensional sensor data**. Since the sensor data contains many features, directly using all features can increase computational complexity and may contain redundant information.

An **Undercomplete Autoencoder (UAE)** is designed to learn a compact latent representation of the sensor data. The encoder compresses the input into a lower-dimensional latent vector, and the decoder reconstructs the original input.

Requirements:

- Input: High-dimensional sensor features
- Output: Reconstructed sensor features
- Latent representation: Lower-dimensional feature vector
- Purpose: Dimensionality reduction and feature learning
- Reconstruction loss: Mean Squared Error (MSE)

2. Proposed Architecture

Example architecture for **100 sensor features**:

Encoder:

- Input layer: 100 neurons
- Dense layer: 64 neurons, ReLU
- Dense layer: 32 neurons, ReLU
- Latent layer: 10 neurons, ReLU

Decoder:

- Dense layer: 32 neurons, ReLU
- Dense layer: 64 neurons, ReLU
- Output layer: 100 neurons, Linear

Architecture Diagram

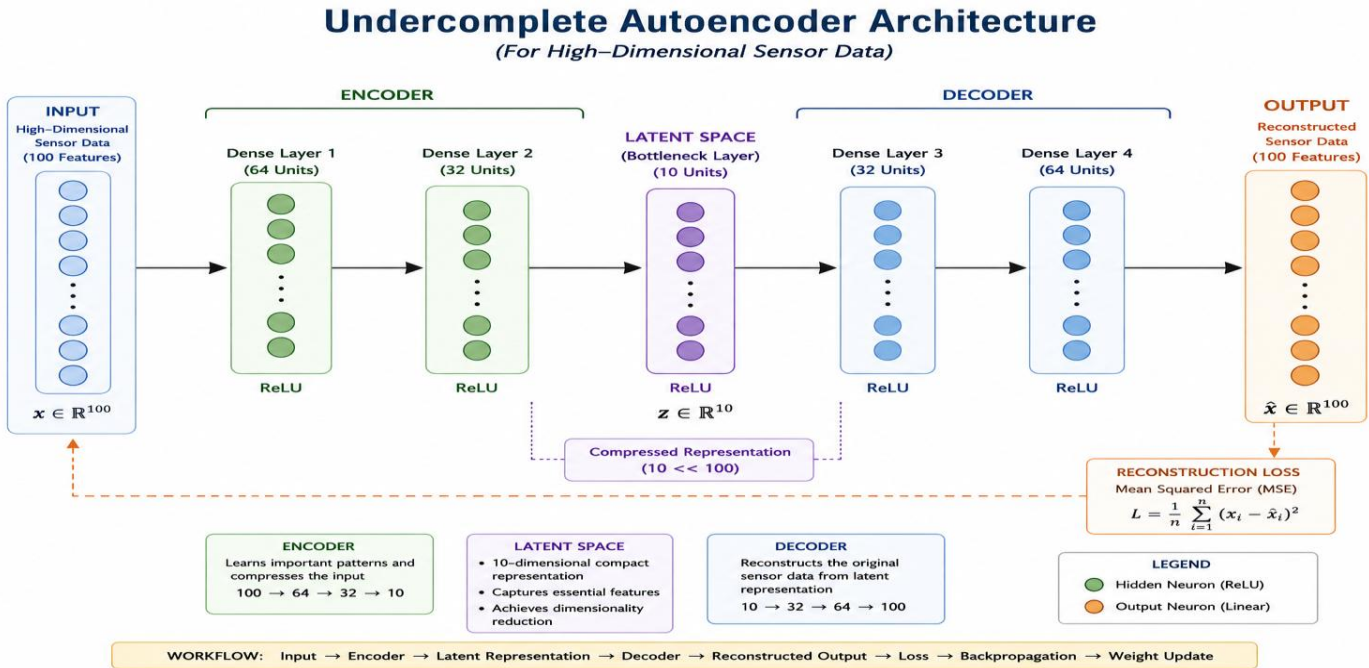


Figure 1: Architecture Diagram of Undercomplete Autoencoder

3. Encoder

The encoder converts the original high-dimensional sensor vector x into a smaller latent representation z .

$$z = f_{\theta}(x)$$

where:

- x = input sensor data
- z = latent representation
- f_{θ} = encoder network

The encoder progressively reduces the number of features:

$$100 \rightarrow 64 \rightarrow 32 \rightarrow 10$$

The **10-dimensional latent vector** contains the most important information learned from the original 100 sensor features.

4. Latent Representation

The latent layer acts as a **compressed representation** of the sensor data.

Instead of processing all 100 features, the system learns only 10 important features.

This provides:

- Dimensionality reduction
- Removal of redundant information
- Lower computational complexity
- Useful feature extraction
- Better representation for subsequent defect-detection models

The architecture is called **undercomplete** because:

$$\text{Latent Dimension} < \text{Input Dimension}$$

Here:

$$10 < 100$$

5. Decoder

The decoder reconstructs the original sensor data from the latent representation:

$$\hat{x} = g_{\phi}(z)$$

where:

- z = latent representation
- $\hat{x}$ = reconstructed input
- g_{ϕ} = decoder network

The decoder expands the representation:

$$10 \rightarrow 32 \rightarrow 64 \rightarrow 100$$

The reconstructed output should be as close as possible to the original sensor input.

6. Activation Functions

Layer	Units	Activation	Purpose
Input	100	—	Sensor features
Encoder 1	64	ReLU	Learn nonlinear features
Encoder 2	32	ReLU	Further feature extraction
Latent	10	ReLU	Compact representation
Decoder 1	32	ReLU	Feature reconstruction
Decoder 2	64	ReLU	Restore representation
Output	100	Linear	Reconstruct sensor values

ReLU is used in hidden layers because it introduces non-linearity and allows the network to learn complex

relationships.

A **linear output activation** is suitable when the sensor values are continuous.

7. Reconstruction Loss

The model can use **Mean Squared Error (MSE)**:

$$L_{MSE} = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{x}_i)^2$$

where:

- x_i = original sensor value
- $\hat{x}_i$ = reconstructed sensor value
- n = number of features

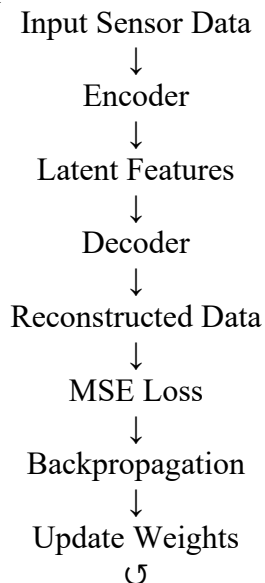
The objective is:

$$\min L_{MSE}$$

Therefore, the autoencoder learns parameters that minimize the difference between the original and reconstructed sensor data.

8. Training Process

1. Sensor data is given to the input layer.
2. The encoder compresses the data into the 10-dimensional latent space.
3. The decoder reconstructs the original 100 features.
4. MSE calculates the reconstruction error.
5. Backpropagation calculates gradients.
6. An optimizer such as **Adam** updates the network weights.
7. The process is repeated for multiple epochs until reconstruction error is minimized.



9. How It Helps in Defect Detection

After training, the **encoder part** can be used to transform the original high-dimensional sensor data into compact features.

For example:

100 sensor features → 10 learned features

These learned features can then be supplied to a classifier such as a neural-network or other classification model to identify **defective and non-defective products**.

The autoencoder therefore performs **feature learning before classification**.

10. Evaluation Measures

The proposed system can be evaluated using:

- **Reconstruction MSE** – measures reconstruction quality.
- **MAE** – measures average reconstruction error.
- **Classification Accuracy** – evaluates defect detection when the latent features are used by a classifier.
- **Precision and Recall** – important for identifying defective products.
- **F1-score** – balances precision and recall.

11. Expected Outcome

The proposed Undercomplete Autoencoder is expected to:

- Reduce the dimensionality of high-dimensional sensor data.
- Learn meaningful hidden features automatically.
- Remove redundant information.
- Reduce computational requirements.
- Provide useful features for defect classification.
- Achieve low reconstruction error while maintaining important information.

Conclusion

An **Undercomplete Autoencoder** is suitable for the manufacturing application because it learns a compact representation by forcing high-dimensional sensor data through a smaller latent space. The encoder performs dimensionality reduction, while the decoder reconstructs the original data. Minimizing reconstruction loss enables the model to learn useful features that can subsequently support **defective-product detection**. This directly addresses the Q1 requirement to design an undercomplete autoencoder and explain its dimensionality-reduction and feature-learning behavior.

Q2. Design a Robust Regularized Autoencoder

1. Problem Understanding

A healthcare organization has **noisy patient-record features** and requires a robust representation for downstream classification. A **Regularized Autoencoder** can learn useful features while reducing the effect of noise and preventing overfitting.

The proposed model uses an **L2 weight regularization** strategy along with the reconstruction objective. The encoder compresses the noisy patient-record features into a latent representation, while the decoder reconstructs the original features.

The design addresses the requirements given in Q2: encoder-decoder structure, regularization mechanism, training objective, reduction of overfitting, and useful feature extraction.

2. Proposed Architecture

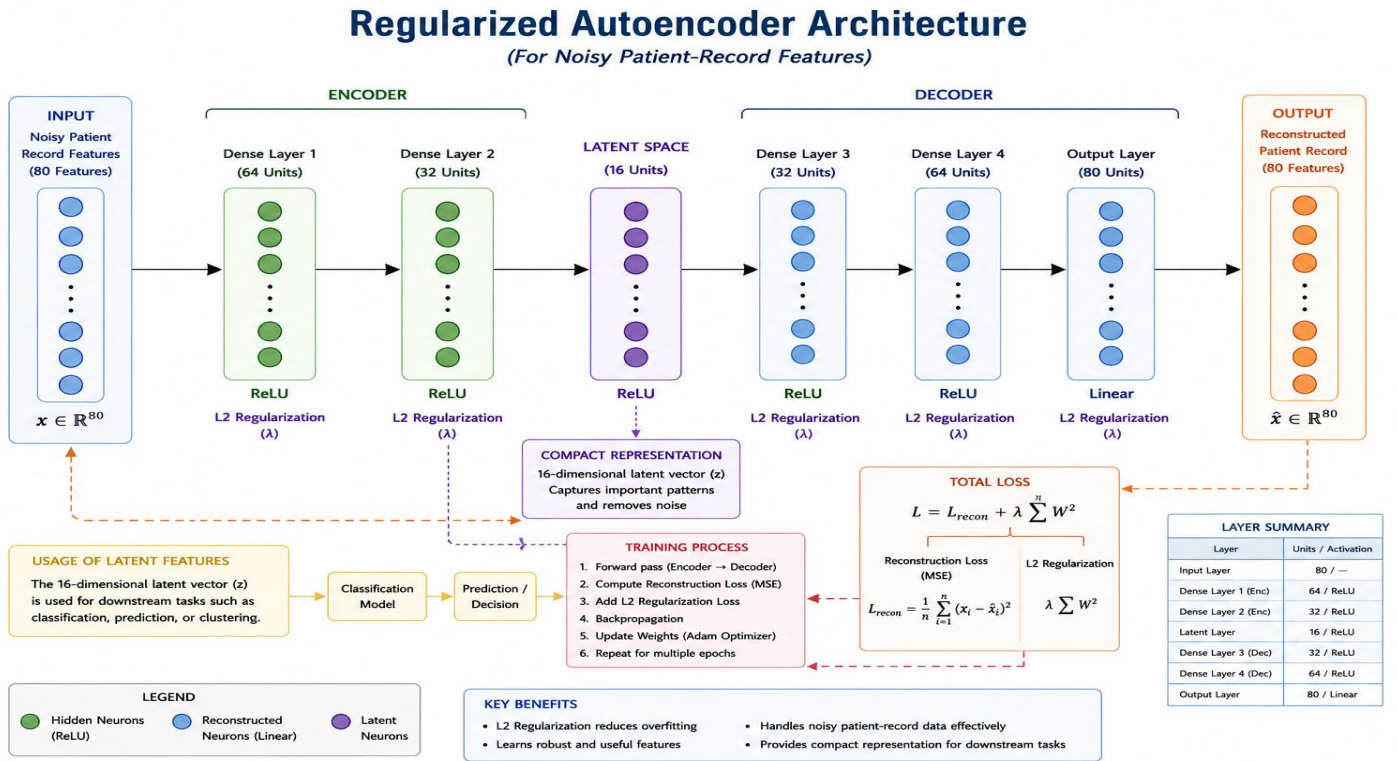
Assume the patient record contains **80 input features**.

Encoder:

- Input: 80 features
- Dense layer: 64 neurons, ReLU
- Dense layer: 32 neurons, ReLU
- Latent layer: 16 neurons, ReLU

Decoder:

- Dense layer: 32 neurons, ReLU
- Dense layer: 64 neurons, ReLU
- Output: 80 neurons, Linear



3. Encoder

The encoder transforms the noisy patient-record vector x into a compact latent representation:

$$z = f_{\theta}(x)$$

The dimensionality is reduced as:

$$80 \rightarrow 64 \rightarrow 32 \rightarrow 16$$

The latent representation contains the important patterns in the patient data while reducing unnecessary information.

4. Regularization Mechanism

The proposed autoencoder uses **L2 weight regularization**.

The regularization term is:

$$L_{reg} = \lambda \sum W^2$$

where:

- W = network weights
- λ = regularization coefficient

The complete training objective becomes:

$$L = L_{reconstruction} + \lambda \sum W^2$$

For continuous patient-record features, reconstruction loss can be **Mean Squared Error (MSE)**:

$$L_{MSE} = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{x}_i)^2$$

Therefore:

$$L = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{x}_i)^2 + \lambda \sum W^2$$

The model therefore tries to reconstruct the input accurately while keeping the network weights from becoming excessively large.

5. How Regularization Reduces Overfitting

Without regularization, the autoencoder may learn the training data too closely, including unwanted patterns and noise.

L2 regularization:

- Penalizes excessively large weights.
- Encourages simpler network representations.
- Reduces dependence on individual features.
- Helps prevent overfitting.
- Improves generalization to unseen patient records.
- Encourages the model to learn more useful patterns.

6. Latent Representation

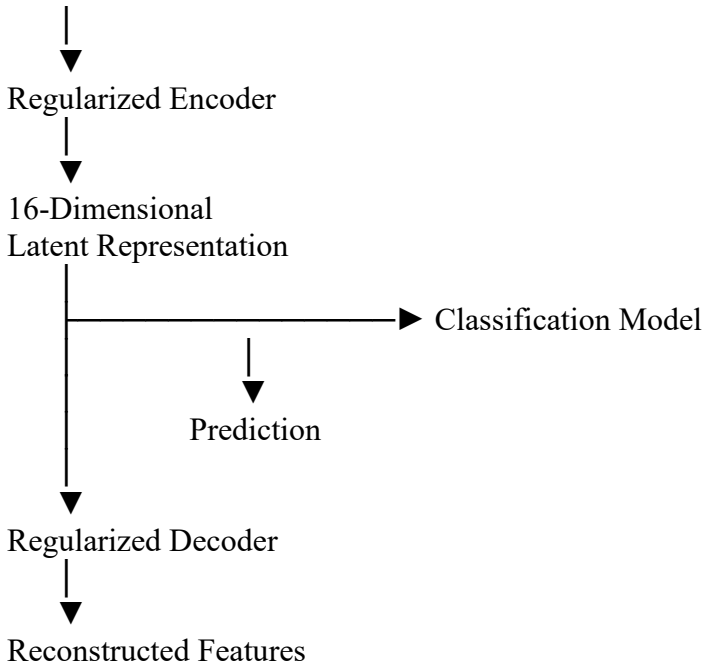
The **16-dimensional latent layer** acts as the compact representation.

$$80 \rightarrow 16$$

Thus, the model reduces the original 80 features to 16 learned features.

These latent features can subsequently be given to a classification model for downstream tasks.

Patient Features



7. Decoder

The decoder reconstructs the patient-record features from the latent representation:

$$\hat{x} = g_{\phi}(z)$$

The dimensionality increases as:

$$16 \rightarrow 32 \rightarrow 64 \rightarrow 80$$

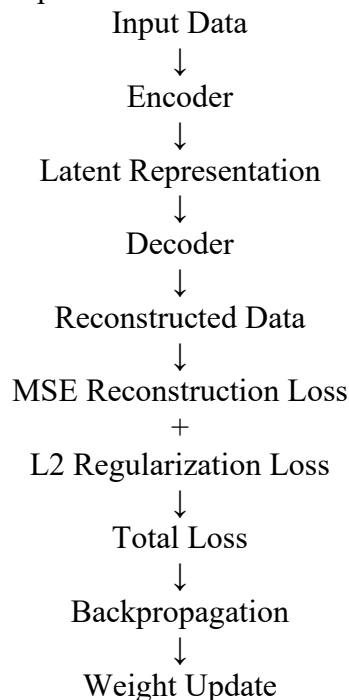
The reconstructed output should be as close as possible to the original input.

8. Activation Functions

Layer	Units	Activation	Purpose
Input	80	—	Patient-record features
Encoder 1	64	ReLU	Nonlinear feature learning
Encoder 2	32	ReLU	Feature compression
Latent	16	ReLU	Compact representation
Decoder 1	32	ReLU	Feature reconstruction
Decoder 2	64	ReLU	Restore features
Output	80	Linear	Reconstruct continuous features

9. Training Process

1. Noisy patient-record data is supplied to the input.
2. The encoder extracts and compresses the features.
3. The latent layer produces the compact representation.
4. The decoder reconstructs the input.
5. MSE calculates reconstruction error.
6. L2 regularization adds a penalty for large weights.
7. The total loss is calculated.
8. Backpropagation calculates gradients.
9. Adam optimizer updates the weights.
10. The process is repeated for multiple epochs.



10. Evaluation Measures

The system can be evaluated using:

- **Reconstruction MSE** – measures reconstruction error.
- **MAE** – measures average reconstruction difference.
- **Classification Accuracy** – evaluates downstream classification.
- **Precision** – measures correctness of positive predictions.
- **Recall** – measures how many relevant cases are detected.
- **F1-score** – balances precision and recall.

11. Expected Outcomes

The proposed Regularized Autoencoder is expected to:

- Learn robust patient-record representations.
- Reduce the effect of irrelevant or noisy patterns.
- Reduce overfitting through L2 regularization.
- Extract useful low-dimensional features.
- Improve generalization to unseen records.
- Provide meaningful latent features for downstream classification.

Conclusion

A **Regularized Autoencoder with L2 weight regularization** is suitable for noisy healthcare data because it combines reconstruction-based feature learning with a penalty that discourages overly complex weight values. The encoder produces a compact 16-dimensional representation from 80 input features, while the decoder reconstructs the original data. The combined reconstruction and regularization objective helps reduce overfitting and produces more useful representations for downstream classification. This satisfies the Q2 requirements specified in the assessment.

Q3. Design a Deep Generative Model — Deep Belief Network (DBN)

1. Problem Understanding

A recommendation/content-generation system needs to learn **complex data distributions** from user preferences, item characteristics, or content features. A **Deep Belief Network (DBN)** is suitable because it consists of multiple hidden layers that learn hierarchical representations of the input data.

For this design, a DBN is proposed for a **recommendation system** that learns hidden patterns in user-item interaction data and can generate or predict suitable recommendations.

The question specifically requires a deep generative model using suitable visible and hidden layers, its architecture, learning process, and justification.

2. Proposed DBN Architecture

Assume the recommendation system has **100 input features** representing user-item preferences.

Architecture

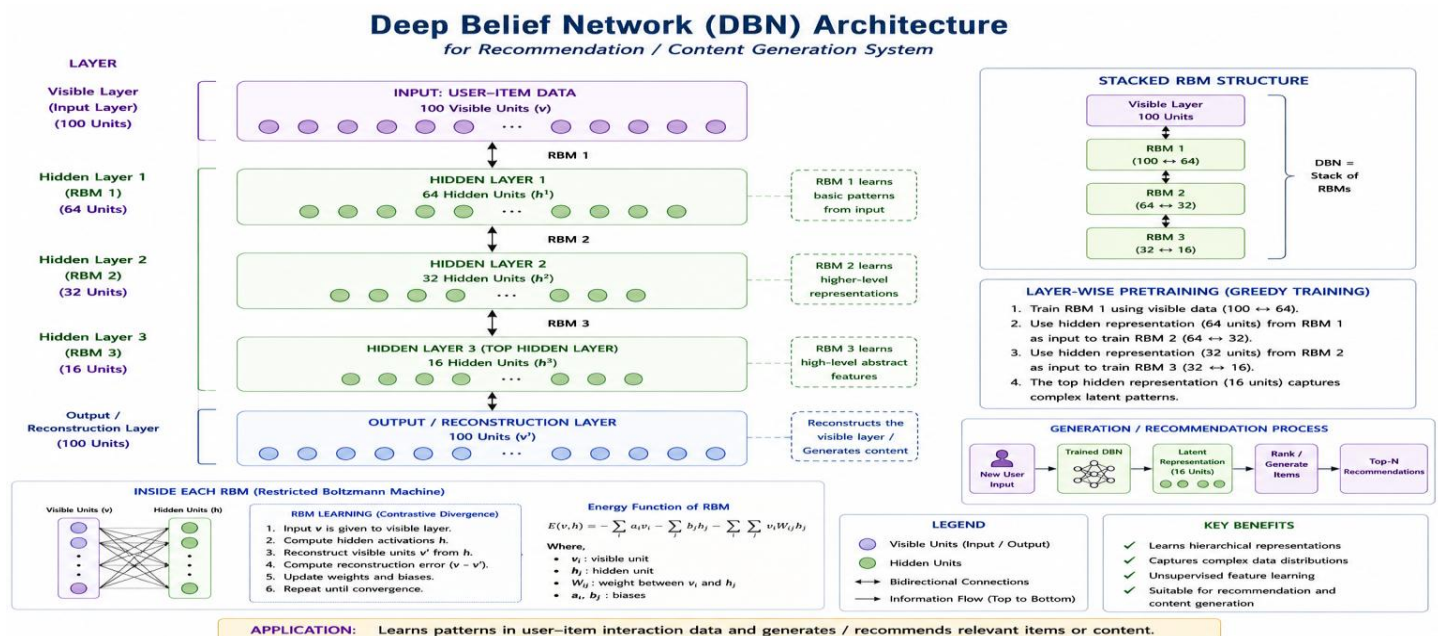


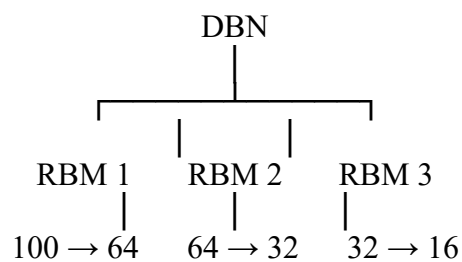
Figure 3: Deep Belief Network (DBN) Architecture

Layer Configuration

Layer	Units	Role
Visible Layer	100	User-item input features
Hidden Layer 1	64	Learns low-level patterns
Hidden Layer 2	32	Learns higher-level relationships
Hidden Layer 3	16	Learns complex latent features
Output	100	Reconstruction/generation

3. DBN Structure

A DBN is constructed by **stacking multiple Restricted Boltzmann Machines (RBMs)**.



Each RBM contains:

- A visible layer
- A hidden layer
- Connections between visible and hidden units
- No connections between units within the same layer

The hidden representation learned by one RBM becomes the input representation for the next RBM.

4. Visible Layer

The visible layer receives the input data.

For example, the 100 features may represent:

- User preferences
- Item ratings
- Purchase history
- Content characteristics
- User-item interactions

The input vector can be represented as:

$$x = [x_1, x_2, x_3, \dots, x_{100}]$$

The DBN uses these observations to discover hidden relationships.

5. Hidden Layers

Hidden Layer 1 — 64 Units

The first hidden layer learns basic relationships from the input data.

$$100 \rightarrow 64$$

It can learn patterns such as:

- Frequently preferred items
- Common user preferences
- Basic item relationships

Hidden Layer 2 — 32 Units

The second layer receives the representation learned by the first layer.

$$64 \rightarrow 32$$

It learns more abstract relationships between users and items.

Hidden Layer 3 — 16 Units

The third layer provides a compact high-level representation.

$$32 \rightarrow 16$$

This layer captures complex latent patterns that can be used for recommendation or generation.

6. Learning Mechanism

The DBN is trained using **layer-wise unsupervised learning**.

Step 1: Train First RBM

The first RBM learns the relationship between the visible layer and Hidden Layer 1.

Visible Data



RBM 1

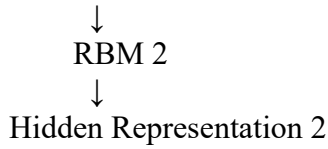


Hidden Representation 1

Step 2: Train Second RBM

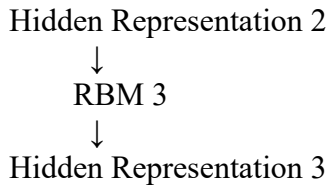
The representation learned from the first RBM is used as input to the second RBM.

Hidden Representation 1



Step 3: Train Third RBM

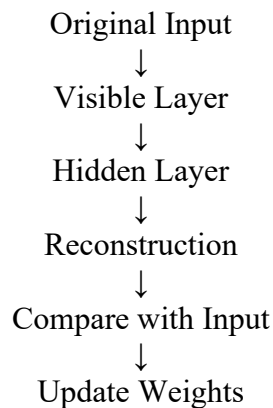
The second hidden representation is passed to the third RBM.



This process is called **greedy layer-wise training**.

7. RBM Learning

An RBM learns by comparing the original input with a reconstructed version.



A common learning approach is **Contrastive Divergence (CD)**.

The basic process is:

1. Input data is given to the visible layer.
2. Hidden-unit activations are calculated.
3. The hidden representation reconstructs the visible layer.
4. The difference between the original and reconstructed data is calculated.
5. Weights are updated.
6. The process is repeated until the model learns useful representations.

8. Energy Function

An RBM can use an energy function to model the relationship between visible and hidden units:

$$E(v, h) = - \sum_i a_i v_i - \sum_j b_j h_j - \sum_i \sum_j v_i W_{ij} h_j$$

where:

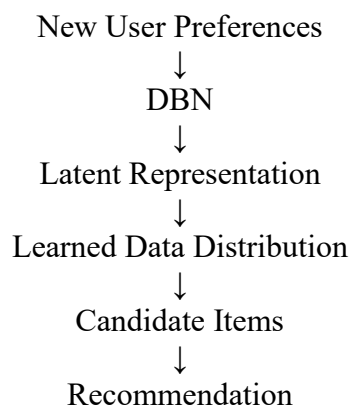
- v_i = visible unit
- h_j = hidden unit
- W_{ij} = weight between visible and hidden units
- a_i = visible bias
- b_j = hidden bias

The model learns parameters that give lower energy to likely configurations of the data.

9. Generation / Recommendation Process

After training, the DBN has learned a representation of the data distribution.

For a new user:



The model can identify patterns similar to previously learned user-item relationships and use them to generate suitable recommendations.

10. Why DBN is Suitable

A DBN is suitable for this application because:

- It learns **hierarchical representations**.
- It can model **complex data distributions**.
- It performs **unsupervised feature learning**.
- Multiple hidden layers capture increasingly abstract patterns.

- It can learn useful latent representations from user-item data.
- The learned representation can support recommendation and content-generation tasks.

11. Evaluation Measures

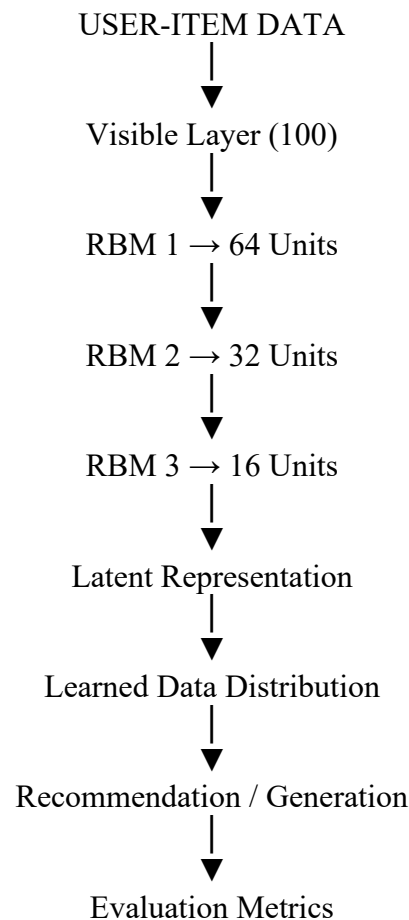
- **Precision@K** – measures relevant recommendations among the top K results.
- **Recall@K** – measures how many relevant items are retrieved.
- **F1-score** – balances precision and recall.
- **RMSE** – useful when predicting numerical ratings.
- **Reconstruction Error** – measures how well the model represents the input.
- **User satisfaction** – evaluates practical recommendation quality.

12. Expected Outcome

The proposed DBN is expected to:

- Learn hidden relationships between users and items.
- Extract hierarchical features automatically.
- Represent complex user preferences using latent variables.
- Generate useful recommendations.
- Improve the quality of content-selection or recommendation decisions.

13. Complete Workflow



Conclusion

A **Deep Belief Network** with three stacked RBMs is proposed for the recommendation/content-generation system. The model transforms 100 input features through **64, 32, and 16 hidden units**, progressively learning higher-level representations. Layer-wise RBM training using **Contrastive Divergence** enables the network to learn complex data distributions without requiring fully labeled training data. The learned latent representation can then be used to generate or recommend suitable content

Q4. Design a Stochastic and Contractive Representation Model

Proposed Model: Contractive Autoencoder (CAE)

1. Problem Understanding

A real-world image or signal-processing application may contain **small variations in the input**, such as minor changes in lighting, sensor noise, position, or signal fluctuations. The representation should remain stable even when these small variations occur.

For this application, a **Contractive Autoencoder (CAE)** is proposed. It learns a compact representation while adding a **contractive penalty** that discourages the encoder from changing significantly for small changes in the input.

The design directly addresses the Q4 requirement to specify the architecture, objective function, robustness mechanism, and explain how representation stability is improved.

2. Proposed Application

Application: Industrial image inspection

The system receives an image of a manufactured product and learns robust features for detecting whether the product is normal or defective.

Requirements

- **Input:** Preprocessed product image
- **Encoder:** Extracts compact features
- **Latent representation:** Stores important image features
- **Decoder:** Reconstructs the input image
- **Contractive penalty:** Makes the representation stable against small input variations
- **Output:** Reconstructed image and robust latent features

3. Proposed Architecture

Assume the input image is resized to **64 × 64 grayscale pixels**.

$$64 \times 64 = 4096$$

Therefore, the input contains **4096 features**.

Architecture

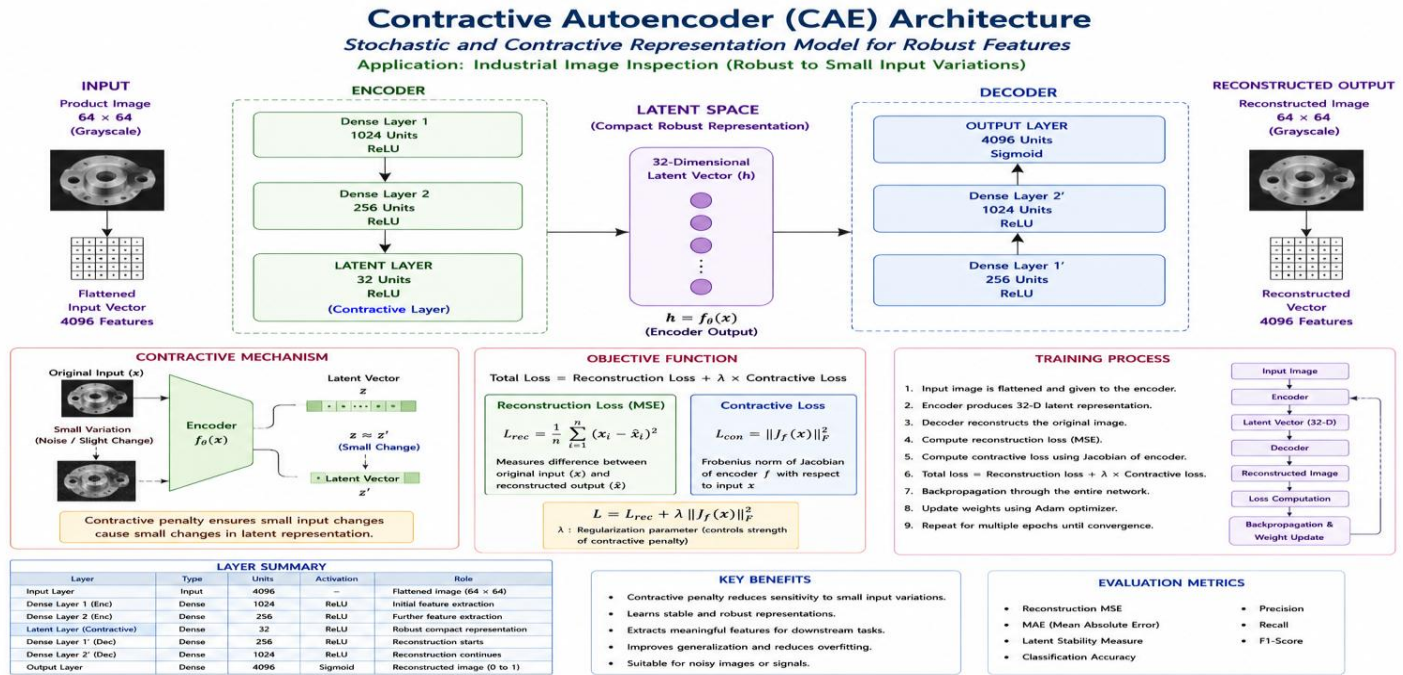


Figure 4: Contractive Autoencoder (CSE) Architecture

4. Encoder

The encoder converts the input image x into a compact latent representation h .

$$h = f_{\theta}(x)$$

The dimensionality is progressively reduced:

$$4096 \rightarrow 1024 \rightarrow 256 \rightarrow 32$$

The **32-dimensional latent vector** captures the important characteristics of the product image.

5. Latent Representation

The latent layer is the most important part of the Contractive Autoencoder.

Input Image



Encoder



32-Dimensional
Latent Representation



Decoder



Reconstructed Image

The latent representation should remain approximately unchanged when the input image undergoes a small variation.

For example:

Original Image → Latent Vector Z



Small Variation



Slightly Changed Image → Latent Vector Z'

The goal is:

$$Z \approx Z'$$

This provides **representation stability**.

6. Contractive Mechanism

The Contractive Autoencoder adds a penalty based on the **Jacobian of the encoder**.

The Jacobian measures how sensitive the latent representation is to changes in the input.

The contractive penalty can be expressed as:

$$L_{contractive} = \|J_f(x)\|_F^2$$

where:

- $J_f(x)$ = Jacobian matrix of the encoder
- $\|\cdot\|_F^2$ = squared Frobenius norm

The model is therefore encouraged to have a small encoder sensitivity.

7. Objective Function

The total objective combines reconstruction loss and the contractive penalty.

$$L = L_{reconstruction} + \lambda \|J_f(x)\|_F^2$$

For image reconstruction, **Mean Squared Error (MSE)** can be used:

$$L_{reconstruction} = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{x}_i)^2$$

Therefore:

$$L = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{x}_i)^2 + \lambda \|J_f(x)\|_F^2$$

Here, λ controls the strength of the contractive penalty.

8. Robustness Mechanism

The contractive penalty forces the encoder to produce representations that are **less sensitive to small input changes**.

Without Contractive Regularization

Small Input Change



Large Latent Change



Unstable Representation

With Contractive Regularization

Small Input Change



Small Latent Change



Stable Representation

Thus, the model learns features that are more robust to small variations.

9. Activation Functions

Layer	Units	Activation	Purpose
Input	4096	—	Image pixels
Encoder 1	1024	ReLU	Feature extraction

Encoder 2	256	ReLU	Feature compression
Latent	32	ReLU	Robust representation
Decoder 1	256	ReLU	Feature reconstruction
Decoder 2	1024	ReLU	Image reconstruction
Output	4096	Sigmoid	Reconstruct normalized pixels

The **ReLU** activation allows nonlinear feature learning, while **Sigmoid** is suitable when the image pixels are normalized between 0 and 1.

10. Decoder

The decoder reconstructs the original image from the latent representation:

$$\hat{x} = g_{\phi}(h)$$

The dimensionality increases as:

$$32 \rightarrow 256 \rightarrow 1024 \rightarrow 4096$$

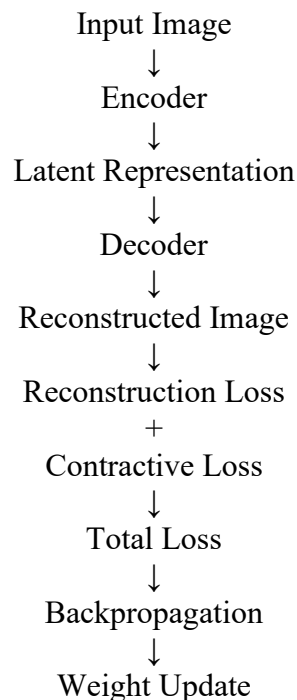
The reconstructed output should be similar to the original image.

11. Training Process

The CAE can be trained using the following process:

1. Preprocess and normalize the product image.
2. Flatten the 64×64 image into 4096 features.
3. Pass the image through the encoder.
4. Generate the 32-dimensional latent representation.
5. Pass the latent representation through the decoder.
6. Reconstruct the original image.
7. Calculate reconstruction loss.
8. Calculate the contractive penalty using the encoder Jacobian.
9. Combine both losses.
10. Apply backpropagation.
11. Update weights using an optimizer such as Adam.
12. Repeat for multiple epochs.

Training Workflow



12. Robust Feature Extraction

After training, the **encoder can be used independently** to extract robust features.

Product Image



Trained Encoder



32-D Robust Features



Classifier



Normal / Defective

This allows the learned representation to be used for downstream classification.

13. Evaluation Measures

The model can be evaluated using:

- **Reconstruction MSE** – measures reconstruction quality.
- **MAE** – measures average reconstruction error.
- **Latent stability** – measures how much the latent representation changes under small input variations.
- **Classification Accuracy** – evaluates defect classification.
- **Precision and Recall** – evaluates detection quality.
- **F1-score** – balances precision and recall.

14. Expected Outcomes

The proposed Contractive Autoencoder is expected to:

- Learn compact image representations.
- Reduce sensitivity to small input variations.
- Produce stable latent features.
- Improve robustness against minor image or signal changes.
- Extract useful features for defect classification.
- Reconstruct the original image with low error.

15. Advantages of the Proposed Design

Feature	Benefit
Compact latent layer	Reduces dimensionality
Contractive penalty	Improves representation stability
Encoder-decoder structure	Learns useful representations
MSE reconstruction loss	Maintains important input information
Jacobian penalty	Reduces sensitivity to small changes
Robust latent features	Useful for downstream classification

Conclusion

A **Contractive Autoencoder** is suitable for image and signal-processing applications where representations must remain stable under small input variations. The proposed architecture compresses a **4096-feature image into a 32-dimensional latent representation** and reconstructs the original image through the decoder. By adding the **Jacobian-based contractive penalty** to the reconstruction loss, the encoder becomes less sensitive to small changes in the input. Therefore, the model provides a stable and robust representation suitable for downstream tasks such as **defect classification**. This fulfills the Q4 requirement for architecture, objective function, robustness mechanism, and representation stability.

Q5. Design a GAN-Based Data Generation System

1. Problem Understanding

An organization has **limited training images** and needs to generate realistic synthetic samples to increase the available training data.

A **Generative Adversarial Network (GAN)** is suitable because it contains two competing neural networks:

- **Generator (G):** Creates synthetic images from random noise.
- **Discriminator (D):** Determines whether an image is real or generated.

The two networks are trained together in an adversarial process. The Generator tries to produce increasingly realistic images, while the Discriminator learns to distinguish real images from fake ones.

The question specifically requires the input, network flow, adversarial training process, loss functions, evaluation approach, and comparison with an Autoencoder.

2. Proposed Application

Application: Synthetic manufacturing-product image generation

Suppose the organization has only a small number of real product images. The GAN generates additional realistic product images that can be used to augment the training dataset.

Requirements

- **Input:** Random noise vector
- **Generator:** Produces synthetic images
- **Discriminator:** Classifies images as real or fake
- **Output:** Realistic synthetic product images
- **Training:** Adversarial learning
- **Evaluation:** Image quality and similarity measures

3. Proposed GAN Architecture

Assume the target image size is 64×64 RGB.

The Generator receives a random noise vector of **100 dimensions**.

Architecture Diagram

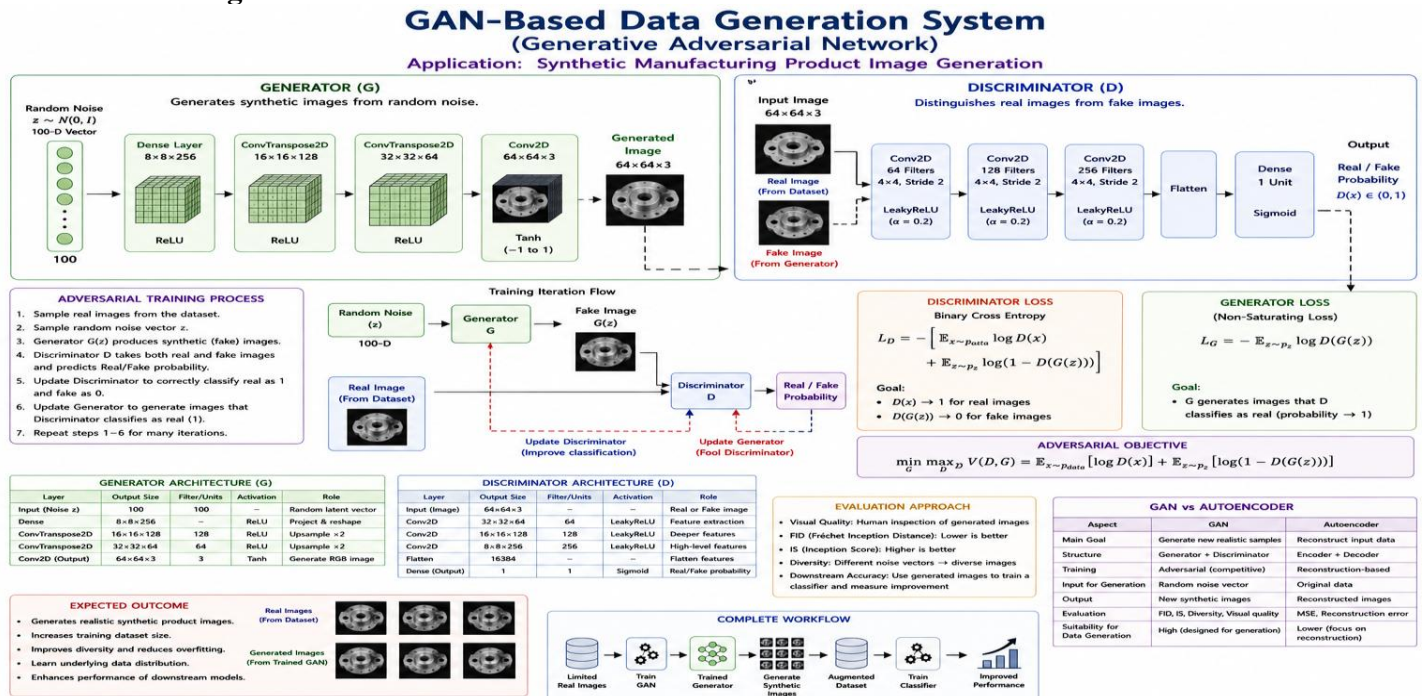


Figure 5: GAN – Based Data Generation System

4. Generator

The Generator G converts a random noise vector into a synthetic image.

$$G(z) = \hat{x}$$

where:

- z = random noise vector
- G = Generator
- $\hat{x}$ = generated image

The Generator architecture is:

$$100 \rightarrow 8 \times 8 \times 256 \rightarrow 16 \times 16 \times 128 \rightarrow 32 \times 32 \times 64 \rightarrow 64 \times 64 \times 3$$

Generator Layers

Layer	Output	Activation	Purpose
Noise Input	100	—	Random latent vector
Dense	$8 \times 8 \times 256$	ReLU	Initial feature generation
ConvTranspose	$16 \times 16 \times 128$	ReLU	Image upsampling
ConvTranspose	$32 \times 32 \times 64$	ReLU	Feature refinement
Conv2D	$64 \times 64 \times 3$	Tanh	Generate RGB image

The **Tanh** output is suitable when the training images are normalized to the range $[-1, 1]$.

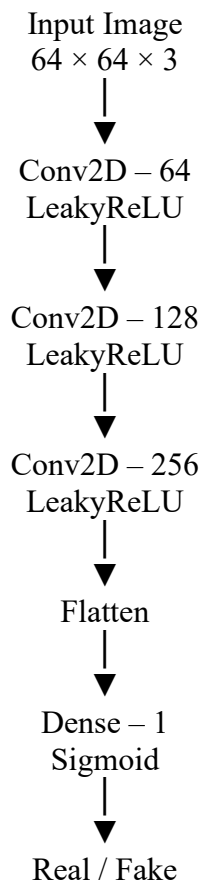
5. Discriminator

The Discriminator receives either a **real image** from the training dataset or a **fake image** produced by the Generator.

Its purpose is to output a probability indicating whether the image is real.

$$D(x) = P(\text{real} \mid x)$$

Discriminator Architecture

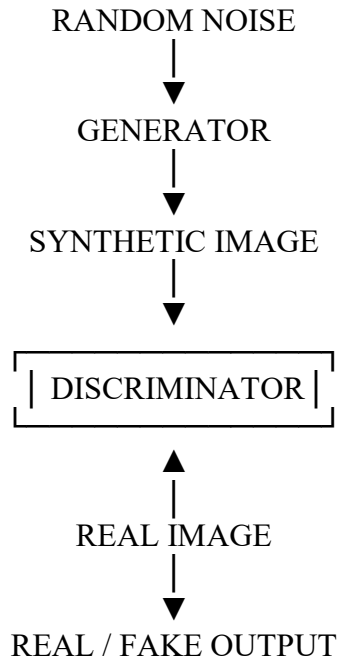


Discriminator Layers

Layer	Units/Filters	Activation	Purpose
Input	$64 \times 64 \times 3$	—	Input image
Conv2D	64	LeakyReLU	Extract image features
Conv2D	128	LeakyReLU	Learn deeper features
Conv2D	256	LeakyReLU	Learn high-level patterns
Flatten	—	—	Convert features to vector
Dense	1	Sigmoid	Real/Fake probability

6. GAN Network Flow

The complete flow is:



The Discriminator receives both:

- Real images from the dataset.
- Fake images generated by the Generator.

7. Adversarial Training Process

GAN training is an iterative competition between the Generator and Discriminator.

Step 1: Generate Fake Images

Random noise is provided to the Generator:

$$z \rightarrow G(z)$$

The Generator produces a synthetic image.

Step 2: Discriminator Receives Real Images

Real training images are given to the Discriminator.

$$x \rightarrow D(x)$$

The Discriminator should classify them as **real**.

Step 3: Discriminator Receives Fake Images

Generated images are given to the Discriminator:

$$G(z) \rightarrow D(G(z))$$

The Discriminator should classify them as **fake**.

Step 4: Update Discriminator

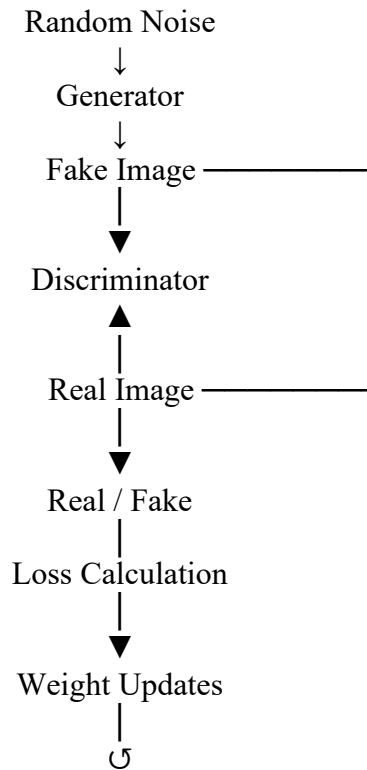
The Discriminator learns to distinguish real images from generated images.

Step 5: Update Generator

The Generator is trained to produce images that the Discriminator classifies as real.

Step 6: Repeat

The process is repeated for many epochs.



8. Discriminator Loss

The standard binary cross-entropy objective for the Discriminator can be written as:

$$L_D = -\left[\log D(x) + \log(1 - D(G(z)))\right]$$

The Discriminator tries to:

- Give high probability to real images.
- Give low probability to generated images.

9. Generator Loss

The Generator tries to make the Discriminator classify generated images as real.

Its non-saturating loss can be written as:

$$L_G = -\log D(G(z))$$

The Generator therefore improves its images based on the feedback received from the Discriminator.

10. Adversarial Objective

The original GAN objective can be expressed as:

$$\min_G \max_D V(D, G) = E_{x \sim p_{data}}[\log D(x)] + E_{z \sim p_z}[\log(1 - D(G(z)))]$$

The two networks have opposing objectives:

Network	Objective
Generator	Generate realistic images
Discriminator	Distinguish real and fake images

As training progresses, the Generator should produce increasingly realistic samples.

11. Training Algorithm

1. Initialize Generator and Discriminator.
2. Select a batch of real images.
3. Generate random noise vectors.
4. Generate fake images using Generator.
5. Train Discriminator using real and fake images.
6. Calculate Discriminator loss.
7. Update Discriminator weights.
8. Generate another batch of fake images.
9. Train Generator to fool the Discriminator.
10. Calculate Generator loss.
11. Update Generator weights.
12. Repeat for multiple epochs.
13. Save the Generator after satisfactory image quality is achieved.

12. Evaluation Approach

The quality of generated images can be evaluated using:

1. Visual Quality

Generated images are visually inspected for:

- Realistic appearance
- Correct product structure
- Absence of obvious artifacts
- Diversity

2. Fréchet Inception Distance (FID)

FID can be used to compare the feature distributions of real and generated images.

Lower FID generally indicates better similarity between real and generated distributions.

3. Inception Score (IS)

Inception Score can evaluate image quality and diversity.

4. Diversity

The generated samples should not all look identical. Different noise vectors should produce diverse images.

5. Downstream Classification Accuracy

Generated images can be added to the training dataset and the improvement in a downstream classifier can be measured.

13. Expected Outcome

The proposed GAN should:

- Generate realistic synthetic product images.
- Increase the size of the training dataset.
- Improve image diversity.
- Reduce the limitation caused by insufficient real training images.
- Provide additional training samples for downstream computer-vision models.
- Learn the underlying distribution of the real product images.

14. GAN vs Autoencoder

Feature	GAN	Autoencoder
Main purpose	Generate realistic new samples	Learn representations/reconstruct inputs
Structure	Generator + Discriminator	Encoder + Decoder
Training	Adversarial learning	Reconstruction-based learning
Input	Random noise for generation	Original data
Output	New synthetic samples	Reconstruction
Image quality	Can produce highly realistic samples	Usually focused on reconstruction

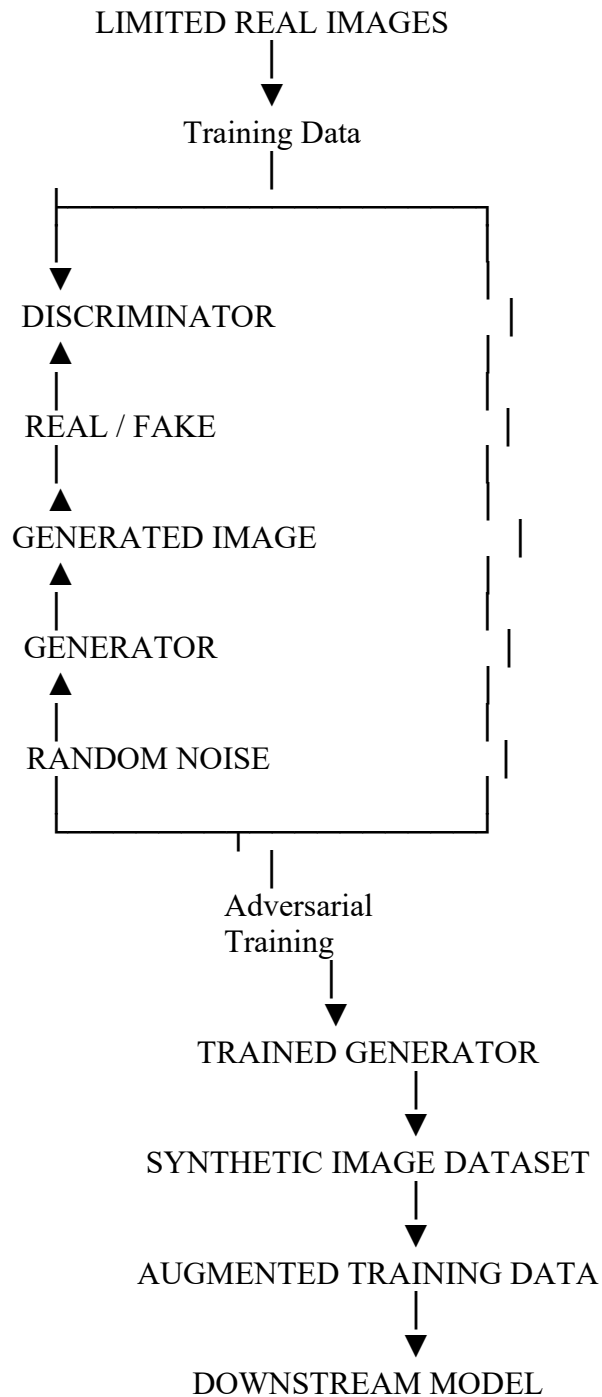
Latent representation	Learned by Generator	Explicit bottleneck representation
Suitable for synthetic image generation	Yes	Less suitable

Why GAN is More Suitable

A GAN is preferred for this application because the primary requirement is to **generate realistic new images** rather than simply reconstruct existing images.

The Generator learns to create new samples, while the Discriminator provides feedback about their realism. Therefore, GAN-based generation is appropriate when an organization has **limited training images and needs realistic synthetic samples**.

15. Complete System Workflow



16. Final Design Summary

Component	Proposed Design
Application	Synthetic product-image generation
Input to Generator	100-dimensional random noise
Generator	Dense + ConvTranspose layers
Generator Output	$64 \times 64 \times 3$ RGB image
Discriminator Input	Real or generated image
Discriminator Output	Real/Fake probability
Generator Activation	ReLU + Tanh
Discriminator Activation	LeakyReLU + Sigmoid
Training	Adversarial training
Generator Loss	$-\log D(G(z))$
Discriminator Loss	Binary cross-entropy
Evaluation	FID, IS, visual quality, diversity, downstream accuracy

Conclusion

The proposed **GAN-based data-generation system** uses a Generator and Discriminator trained adversarially to create realistic synthetic product images. The Generator converts a 100-dimensional random noise vector into a $64 \times 64 \times 3$ image, while the Discriminator distinguishes real images from generated images. Through repeated adversarial training, the Generator learns the distribution of real product images and can produce additional synthetic samples. Compared with an Autoencoder, the GAN is more suitable because the main objective is **realistic new-data generation**.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 Exemplary	Level 3 Proficient	Level 2 Developing	Level 1 Beginning
Understanding of Problem Context	20	Clearly understands the problem, requirements, constraints, and expected outcomes.	Good understanding with minor gaps in requirements.	Basic understanding; some requirements are unclear.	Poor understanding or misinterpretation of the problem.
Application of Concepts	30	Accurately applies appropriate generative deep learning concepts and architectures.	Applies relevant concepts correctly with minor errors.	Applies basic concepts with limited accuracy.	Fails to apply appropriate concepts.
Design Approach	20	Innovative, logical, well-structured, feasible, and technically justified design.	Logical and feasible design with minor gaps.	Basic design with some inconsistencies.	Unclear, illogical, or impractical design.
Accuracy of Solution	20	Completely correct architecture, process, objectives, and justification.	Mostly correct with minor technical errors.	Partially correct solution with noticeable gaps.	Incorrect or incomplete solution.
Clarity	10	Clear, well-organized diagram and explanation with proper technical labeling.	Understandable with minor clarity issues.	Limited clarity and explanation.	Poorly presented and difficult to understand.